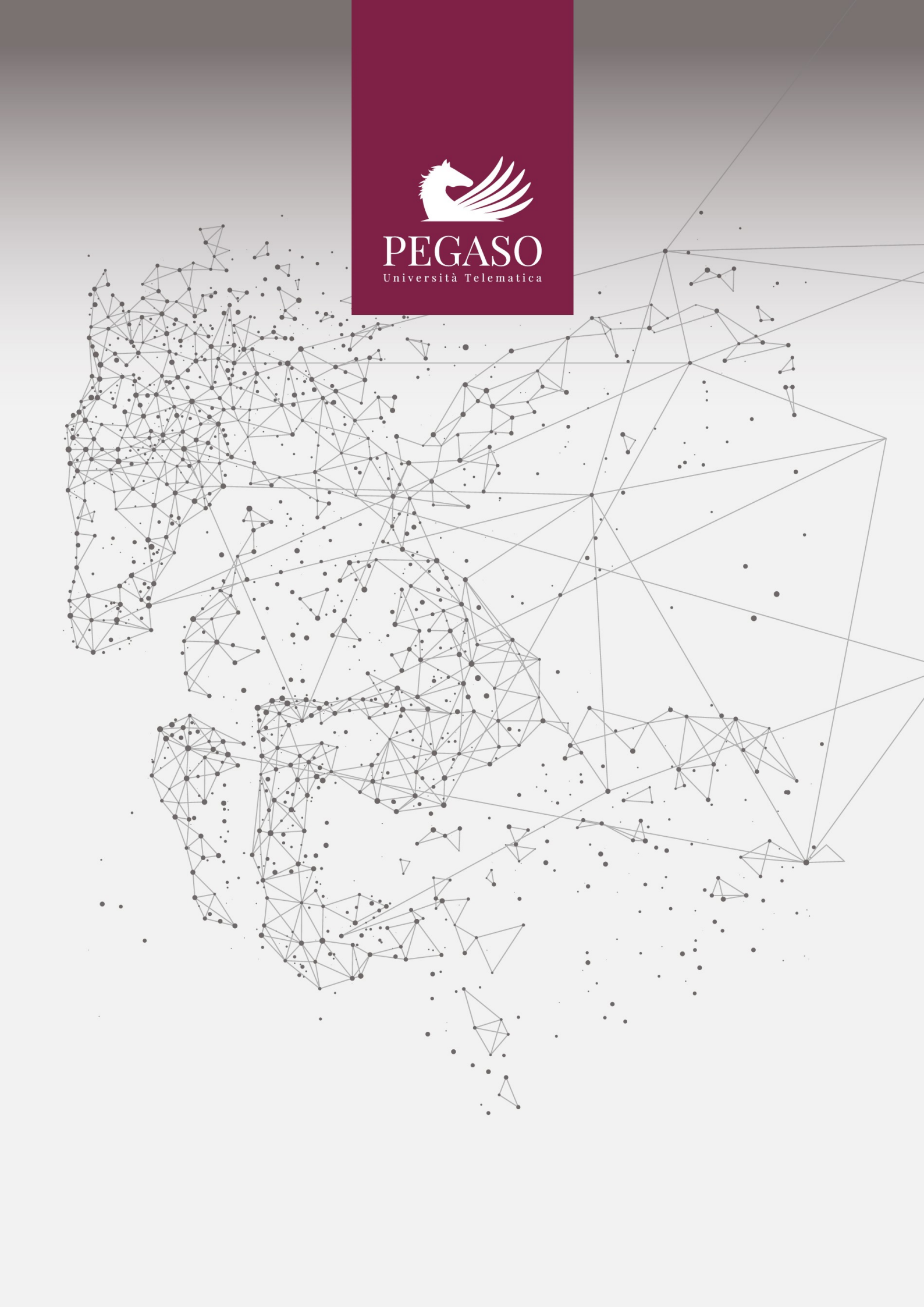

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ANGULARJS 3
2. MODULO – CONTROLLER - DIRETTIVA 5
3. PRIMO ESEMPIO 8
4. NG-MODEL ED NG-BIND 10
BIBLIOGRAFIA 13
SITOGRAFIA 14

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Angularjs

Angular è un framework per lo sviluppo di applicazioni web basate su JavaScript/TypeScript. Creato da Google, Angular fornisce un insieme di strumenti e librerie per semplificare lo sviluppo di applicazioni web complesse e scalabili.

Angular utilizza il pattern architetturale MVC (Model-View-Controller) per organizzare il codice e semplificare la gestione della logica dell'applicazione:

- il Model rappresenta i dati dell'applicazione;
- la View è responsabile della presentazione dei dati;
- il Controller, chiamato "Component", gestisce la logica dell'applicazione e la comunicazione tra Model e View.

Angular utilizza anche i servizi per gestire la logica di business dell'applicazione e le dipendenze tra i diversi componenti.

Tra le principali caratteristiche di Angular ci sono:

- **Modularità**: Angular suddivide le applicazioni in moduli, ognuno dei quali ha uno scopo specifico e può essere riutilizzato in altre applicazioni.
- **Binding dei dati**: Angular offre un sistema di binding dei dati bidirezionale, che consente di collegare i dati del Model alla View e di aggiornare automaticamente i dati quando vengono modificati.
- **Routing**: Angular offre un sistema di routing per la navigazione tra le diverse pagine dell'applicazione.
- **Testing**: Angular offre una vasta gamma di strumenti e librerie per testare l'applicazione e garantire la qualità del codice.
- **Animazioni**: Angular offre un sistema di animazioni che permette di aggiungere effetti visivi all'interfaccia utente.

Angular è scritto in TypeScript, un linguaggio di programmazione basato su JavaScript che fornisce una serie di funzionalità aggiuntive, come la tipizzazione statica e la gestione delle classi. TypeScript viene poi compilato in JavaScript, che può essere eseguito dai browser web moderni.

La CDN (Content Delivery Network) di AngularJS è un servizio che permette di includere la libreria AngularJS all'interno della propria pagina web tramite un URL fornito dal provider della CDN, senza dover scaricare la libreria e installarla localmente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'utilizzo della CDN di AngularJS presenta alcuni vantaggi:

- **Velocità di caricamento**: la CDN di AngularJS è stata progettata per offrire un caricamento veloce della libreria AngularJS. Inoltre, essendo distribuita in diversi server, la CDN permette di scaricare la libreria da un server vicino alla posizione dell'utente, migliorando ulteriormente la velocità di caricamento
- **Riduzione del carico del server**: utilizzando la CDN di AngularJS, il carico del server web diminuisce in quanto la libreria viene caricata dal server della CDN e non dal tuo server.
- **Semplificazione della gestione delle versioni**: la CDN di AngularJS offre la possibilità di specificare la versione della libreria da caricare tramite l'URL, semplificando la gestione delle versioni della libreria.

Per utilizzare la CDN di AngularJS, è sufficiente includere il seguente URL nella pagina HTML:

```
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js
"></script>
```

In questo modo, la libreria AngularJS verrà caricata dal server della CDN di Google.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Modulo – Controller - Direttiva

Il **modulo** è un contenitore che raggruppa gli elementi dell'applicazione Angular, come i controller, le direttive, i filtri, ecc. Un modulo viene definito utilizzando la funzione angular.module() e può essere considerato come un'unità logica dell'applicazione che fornisce un contesto per gli elementi all'interno di esso.

Ad esempio, possiamo definire un modulo "myApp" nel seguente modo:

```
var app = angular.module('myApp', []);
```

In questo caso, abbiamo definito un modulo chiamato myApp utilizzando la funzione angular.module() e passando come secondo parametro un array vuoto che indica che il modulo non ha dipendenze esterne.

Il **controller** è una funzione JavaScript che gestisce il modello dell'applicazione e fornisce i dati e le funzionalità necessarie alla vista: un controller viene definito come una funzione che accetta come parametro l'oggetto $scope, che rappresenta il contesto di esecuzione del controller.

Ad esempio, possiamo definire un controller myCtrl all'interno del modulo myApp nel seguente modo:

```
app.controller('myCtrl', function($scope) {
  $scope.name = "Mondo";
});
```

In questo caso, abbiamo definito un controller myCtrl all'interno del modulo myApp utilizzando la funzione app.controller() e passando come secondo parametro una funzione che definisce il comportamento del controller. Il controller utilizza l'oggetto $scope per definire il modello dell'applicazione, in questo caso una variabile chiamata name.

AngularJS utilizza la sintassi delle doppie parentesi graffe ({{}}) per l'interpolazione delle stringhe: questa sintassi permette di visualizzare il valore di una proprietà del controller all'interno del contenuto HTML.

La **direttiva** è un'etichetta HTML che definisce il comportamento della vista e permette di estendere le funzionalità degli elementi HTML. Una direttiva viene definita utilizzando la funzione app.directive() e può essere considerata come un componente riutilizzabile che può essere utilizzato all'interno di una vista.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, possiamo definire una direttiva "helloWorld" all'interno del modulo "myApp" nel seguente modo:

```
app.directive('helloWorld', function() {
  return {
    template: '<p>Hello World!</p>'
  };
});
```

In questo caso, abbiamo definito una direttiva helloWorld all'interno del modulo myApp utilizzando la funzione app.directive() e passando come secondo parametro un oggetto che definisce il comportamento della direttiva. In questo caso, abbiamo definito una direttiva che visualizza un messaggio di saluto "Hello World!" all'interno di un elemento <p>.

Per utilizzare la direttiva "helloWorld" all'interno di una vista, è possibile utilizzare l'etichetta HTML "hello-world" come segue:

```
<hello-world></hello-world>
```

In questo modo, l'elemento <p> contenente il messaggio "Hello World!" viene generato automaticamente all'interno della vista. La direttiva può essere utilizzata come qualsiasi altro elemento HTML e può essere posizionata ovunque nella vista in base alle esigenze dell'applicazione.

In generale, le direttive sono utilizzate per estendere le funzionalità degli elementi HTML e possono essere utilizzate per creare componenti personalizzati riutilizzabili all'interno dell'applicazione. Le direttive possono anche essere utilizzate per definire comportamenti dinamici della vista in base al modello dell'applicazione, rendendo l'interfaccia utente più interattiva e responsiva.

Angular offre un set di direttive integrate che sono incluse nella libreria base di Angular: queste direttive integrano funzionalità utili per la gestione di casi d'uso comuni, come la gestione dei dati dell'utente, la gestione delle iterazioni e così via.

Ecco un elenco delle principali direttive di AngularJS:

- **ng-app**: specifica il punto di partenza dell'applicazione AngularJS. Viene utilizzata come attributo sull'elemento HTML radice della pagina.
- **ng-init**: consente di inizializzare il valore di una variabile del quando si carica la pagina HTML
- **ng-controller**: specifica il controller da utilizzare per una sezione specifica della pagina HTML. Viene utilizzata come attributo sull'elemento HTML che contiene la sezione.
- **ng-model**: collega l'input dell'utente all'applicazione AngularJS, creando un collegamento bidirezionale tra il modello e la UI. Viene utilizzata come attributo sull'elemento HTML che rappresenta l'input (ad esempio <input> o <textarea>).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **ng-bind**: collega un elemento HTML a una proprietà del controller, in modo che il valore della proprietà venga visualizzato nell'elemento HTML. Viene utilizzata come attributo sull'elemento HTML che rappresenta il contenuto (ad esempio o).
- **ng-repeat**: itera su un array o su un oggetto e crea una copia della sezione HTML per ogni elemento nell'array o nell'oggetto. Viene utilizzata come attributo sull'elemento HTML che contiene la sezione che deve essere ripetuta.

Queste sono solo alcune delle direttive disponibili in AngularJS. Ci sono molte altre direttive, come ng-if, ng-show, ng-hide, ng-click, ng-submit, ng-style, ng-class, ng-options, solo per citarne alcune.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Primo Esempio

Ecco un primo esempio:

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <meta charset="utf-8">
  <title>My AngularJS App</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js
"></script>
  <script src="script.js"></script>
</head>
<body ng-controller="myCtrl">
  <h1>{{ greeting }}</h1>
  <input type="text" ng-model="name">
  <p>Hello, {{ name }}!</p>
</body>
</html>
```

In questo esempio abbiamo definito un'applicazione AngularJS con il nome myApp, utilizzando la direttiva ng-app. Abbiamo quindi incluso la libreria AngularJS tramite la direttiva <script> che carica la versione 1.8.2 di AngularJS dalla CDN di Google.

Abbiamo poi creato un controller chiamato myCtrl in un file app.js separato, che verrà incluso in seguito. All'interno del body della pagina HTML abbiamo definito la direttiva ng-controller per associare il controller myCtrl alla sezione HTML.

Infine, abbiamo utilizzato la direttiva ng-model per creare un collegamento bidirezionale tra l'input utente e il modello AngularJS, e la direttiva {{}} per l'interpolazione delle stringhe, per visualizzare il contenuto dell'input.

Ecco il contenuto del file script.js:

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.greeting = 'Welcome to my AngularJS app!';
});
```

In questo esempio, abbiamo definito un controller chiamato myCtrl utilizzando la funzione app.controller() fornita da AngularJS. Il controller accetta l'oggetto $scope come parametro nella sua funzione di definizione, che viene fornito automaticamente da AngularJS tramite dependency injection. Il controller definisce le proprietà e i metodi dello $scope per comunicare con l'interfaccia utente; in questo

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

modo, lo $scope può essere utilizzato per definire il comportamento dell'interfaccia utente e le sue proprietà possono essere visualizzate nella pagina HTML tramite l'interpolazione delle stringhe {{ greeting }}.

Quando l'applicazione viene eseguita nel browser, l'utente vedrà un input di testo vuoto e una stringa di saluto nella pagina. L'utente può inserire il proprio nome nell'input e il nome verrà visualizzato nella stringa di saluto grazie all'interpolazione delle stringhe {{ name }}.

L'effetto è il seguente:



Figura 1 *- angular test*

Quello che si scrive nell'input text viene inserito automaticamente nel "greeting".

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Ng-model ed ng-bind

La direttiva ng-model è una delle direttive principali di Angular, che permette di creare un legame bidirezionale tra la vista (HTML) e il modello (JavaScript): in altre parole, permette di sincronizzare i dati tra l'input dell'utente e il modello dell'applicazione.

Per utilizzare la direttiva ng-model, è necessario prima definire una variabile nel controller dell'applicazione Angular; questa variabile rappresenta il valore iniziale del dato da visualizzare nell'interfaccia utente e viene poi aggiornata in base all'input dell'utente. Ad esempio, supponiamo di voler creare un campo di input per l'email dell'utente:

```
<div ng-app="myApp" ng-controller="myCtrl">
  <label for="emailInput">Email:</label>
  <input type="email" id="emailInput" ng-model="user.email">
  <p>La tua email è: {{user.email}}</p>
</div>
```

In questo caso, abbiamo utilizzato la direttiva "ng-model" per collegare il campo di input alla variabile "user.email": questo significa che ogni volta che l'utente inserisce un valore nell'input, il valore della variabile "user.email" viene aggiornato automaticamente grazie al javascript associato:

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.user = {email: ""};
});
```

È importante notare che la direttiva ng-model funziona solo con gli elementi di input, come ad esempio <input>, <textarea> e <select>.

La differenza principale tra la direttiva ng-model e la direttiva ng-bind è che la prima permette di creare un legame bidirezionale tra la vista e il modello dell'applicazione, mentre la seconda permette solo di visualizzare il valore del modello nella vista: in altre parole, ng-model permette di sincronizzare automaticamente i dati tra l'input dell'utente e il modello dell'applicazione, mentre ng-bind consente solo di visualizzare i dati dal modello nella vista.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, se abbiamo un campo di input testuale che rappresenta il nome dell'utente, possiamo utilizzare la direttiva ng-model per legare il valore dell'input alla variabile user.name nel modello dell'applicazione in maniera del tutto simile a quanto fatto in precedenza:

```
<div ng-app="myApp" ng-controller="myCtrl">
  <label for="nameInput">Nome:</label>
  <input type="text" id="nameInput" ng-model="user.name">
  <p>Ciao {{user.name}}!</p>
</div>
```

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.user = {name: ""};
});
```

In questo modo, ogni volta che l'utente inserisce un valore nell'input, il valore della variabile user.name viene automaticamente aggiornato.

D'altra parte, se vogliamo visualizzare solo il valore della variabile "user.name" nella vista, possiamo utilizzare la direttiva ng-bind:

```
<p>Il tuo nome è: <span ng-bind="user.name"></span></p>
```

In questo caso, il valore della variabile user.name viene visualizzato nella stringa di interpolazione {{user.name}} nella vista.

In sintesi, ng-model viene utilizzata per gestire la sincronizzazione dei dati tra l'input dell'utente e il modello dell'applicazione, mentre ng-bind viene utilizzata solo per visualizzare i dati dal modello nella vista.

Da precisare che la direttiva ng-bind può essere applicata a qualsiasi elemento HTML che supporta il contenuto testuale, come ad esempio <p>, , <h1>, , ecc. Ad esempio, se abbiamo una variabile "greeting" nel controller dell'applicazione Angular, possiamo utilizzare la direttiva "ng-bind" per visualizzarne il valore in un elemento <p>:

```
<p ng-bind="greeting"></p>
```

In questo modo, il valore della variabile "greeting" viene visualizzato all'interno dell'elemento <p> nella vista. La direttiva "ng-bind" può anche essere utilizzata per visualizzare espressioni complesse nella vista, ad esempio:

```
<p>Il risultato di 5 + 5 è: <span ng-bind="5 + 5"></span></p>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo caso, l'espressione "5 + 5" viene valutata e il risultato viene visualizzato all'interno dell'elemento nella vista.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).